

Container Security Laboratory

Escape da container tramite esposizione del socket Docker
e catena d'attacco completa

Mele Maria Luisa
Corso di Studi: Cloud Computing per l'Intelligenza Artificiale
Materia: Sicurezza per il cloud

Sommario

Questa relazione documenta un laboratorio riproducibile che dimostra un *container escape* basato su una configurazione errata ma estremamente comune: l'esposizione del socket del daemon Docker all'interno di un container applicativo. A partire da un'applicazione web deliberatamente vulnerabile a *command injection*, si ricostruisce l'intera progressione dell'attacco, dallo sfruttamento applicativo fino all'ottenimento di privilegi di root sul sistema ospitante. Rispetto a una semplice descrizione, ogni fase operativa — predisposizione dell'ambiente, sfruttamento, verifica, mitigazione — è qui presentata come una sequenza di passi espliciti e riproducibili. Il lavoro analizza inoltre il meccanismo di isolamento dei container, spiega perché la configurazione scelta ne annulla il confine di sicurezza, discute le vulnerabilità note correlate e propone un insieme di contromisure. L'ambiente è interamente ricostruibile dai file allegati ed è pensato esclusivamente per l'esecuzione in un contesto isolato e monouso.

1 Introduzione e obiettivi

L'adozione dei container ha diffuso l'idea, in parte fuorviante, che un processo in esecuzione dentro un container sia automaticamente "isolato" dall'host. In realtà l'isolamento offerto da Docker è il risultato di una combinazione di meccanismi del kernel Linux che possono essere indeboliti, intenzionalmente o per disattenzione, da singole scelte di configurazione. L'obiettivo di questo laboratorio è mostrare in modo concreto e riproducibile come una di queste scelte — montare il socket del daemon Docker in un container — sia sufficiente ad annullare completamente il confine tra container e host.

Lo scenario non si limita a dimostrare la tecnica in modo astratto: ricostruisce una *catena d'attacco* completa e realistica, in cui l'accesso iniziale al container avviene sfruttando una vulnerabilità applicativa. Per rendere il percorso effettivamente seguibile, la relazione presenta ciascuna fase come una procedura passo passo, indicando per ogni passo l'azione da compiere e il risultato atteso. Questa impostazione consente di ragionare non solo sul singolo escape, ma su come più confini di sicurezza vengano attraversati in sequenza e su quali punti della catena sia più efficace intervenire.

2 Architettura del laboratorio

L'ambiente è composto da un unico servizio, una finta *Container Operations Dashboard*, definito tramite `docker-compose`. L'immagine viene costruita da una piccola applicazione Flask e, oltre all'app, contiene il client Docker: questa combinazione rende verosimile lo scenario, perché è

proprio il tipo di strumento operativo che, per gestire container, viene talvolta avviato con il socket del daemon montato al suo interno. Il bind mount del socket è l'unica configurazione pericolosa introdotta deliberatamente; tutto il resto è configurazione predefinita.

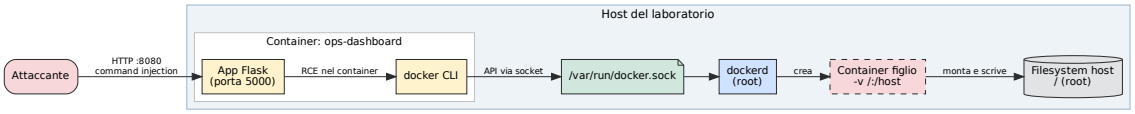


Figura 1: Architettura del laboratorio. L'attaccante raggiunge l'applicazione via HTTP; attraverso il socket montato, un processo nel container chiede al daemon (root) di creare un container figlio che monta il filesystem dell'host.

La definizione del servizio è volutamente minimale. L'elemento critico è il volume che collega il socket dell'host al container:

Listing 1: environment/docker-compose.yml (estratto)

```yaml
services:
  dashboard:
    build: ../application
    image: ops-dashboard:lab
    container_name: ops-dashboard
    ports:
      - "8080:5000"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
```

3 Il meccanismo di isolamento dei container

Per comprendere perché la configurazione scelta sia così dannosa occorre richiamare come Docker realizza l'isolamento. Un container non è una macchina virtuale: non esiste un kernel separato né un confine hardware. Un container è semplicemente un insieme di processi che condividono il kernel dell'host ma ne vedono una "porzione" ristretta, grazie a diversi meccanismi cooperanti.

I **namespace** del kernel creano viste separate delle risorse di sistema: il namespace dei PID nasconde i processi dell'host, quello di mount fornisce un filesystem radice dedicato, quello di rete isola le interfacce, e così via. I **cgroups** limitano e contabilizzano l'uso di risorse come CPU e memoria, impedendo che un container ne monopolizzi. Le **capabilities** frammentano i privilegi tradizionali di root in unità più piccole, in modo che un processo "root" dentro il container possegga in realtà solo un sottoinsieme ridotto di poteri. Infine i profili **seccomp** e i moduli di sicurezza come **AppArmor** o **SELinux** filtrano le chiamate di sistema e le operazioni consentite. È la sovrapposizione di questi strati a produrre ciò che percepiamo come isolamento. [5]

Un punto spesso trascurato è che l'architettura di Docker è di tipo client/daemon. Il comando `docker` è solo un client: il lavoro effettivo è svolto dal processo `dockerd`, che gira come root sull'host ed espone la propria API su un socket di dominio Unix, per impostazione predefinita `/var/run/docker.sock`. Questo socket è il vero centro di controllo del sistema di container. [2]

4 La misconfigurazione: esposizione del socket Docker

La configurazione su cui si fonda il laboratorio consiste nel montare `/var/run/docker.sock` dall'host dentro il container applicativo. Si tratta di una pratica diffusa in contesti reali — pipeline di integrazione continua, agenti di monitoraggio, dashboard di amministrazione — perché apparentemente innocua e molto comoda: consente al servizio di parlare con il daemon per elencare, avviare o fermare container.

Il problema è che il socket non prevede alcun controllo di autorizzazione a grana fine. Come ricorda esplicitamente la *Docker Security Cheat Sheet* di OWASP, concedere accesso a `/var/run/docker.sock` equivale a concedere accesso root illimitato all'host, e montare il socket in sola lettura non è una soluzione, ma rende soltanto più difficile lo sfruttamento. Il daemon esegue qualunque richiesta valida provenga da chiunque sia in grado di scrivere sul socket, con i propri privilegi di root. [1]

5 Perché la configurazione indebolisce il confine di isolamento

Gli strati di isolamento descritti nella Sezione 3 agiscono sul processo del container: ne limitano la vista, i privilegi e le chiamate di sistema. L'abuso del socket, però, non tenta di forzare nessuno di questi strati. Il processo nel container non evade il proprio namespace né eleva le proprie capabilities: si limita a *chiedere al daemon*, che vive al di fuori di tutti quegli strati, di compiere un'azione privilegiata per suo conto. La richiesta tipica è la creazione di un nuovo container che monta il filesystem dell'host.

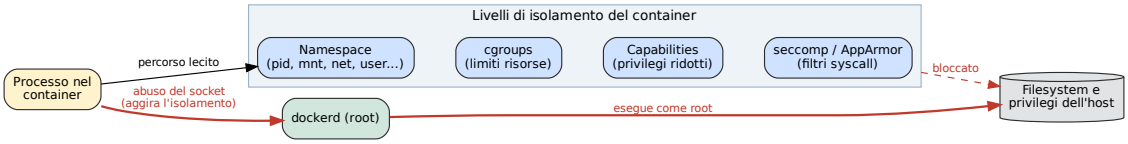


Figura 2: Il percorso "lecito" di un processo del container è vincolato dagli strati di isolamento. L'abuso del socket aggira completamente tali strati: la richiesta raggiunge il daemon, che agisce come root sull'host.

In questo senso non si verifica, tecnicamente, una "fuga" dal container: il confine viene piuttosto *reso irrilevante*. Il container ottiene, per interposta persona, gli stessi poteri del daemon. È questa la ragione per cui l'attacco non richiede il flag `--privileged`, capabilities particolari né alcun exploit del kernel: la delega di privilegio è implicita nella semplice visibilità del socket.

6 Assunzioni e prerequisiti

La riuscita dello scenario presuppone un insieme preciso di condizioni, tutte realistiche:

- il socket Docker è montato in lettura/scrittura nel container applicativo;
- esiste un modo per eseguire comandi all'interno del container (qui una vulnerabilità applicativa raggiungibile via rete);
- il daemon Docker gira come root, ossia nella modalità predefinita e non *rootless*;

- non sono attive contromisure specifiche quali un socket proxy, il remapping degli user namespace o profili seccomp/AppArmor restrittivi.

Vale la pena sottolineare che l'esecuzione come utente non privilegiato *dentro* il container non costituisce di per sé una protezione: se il processo può leggere e scrivere sul socket, l'escalation verso root sull'host resta possibile. La riduzione dei privilegi applicativi, pur buona pratica, non mitiga questa specifica misconfigurazione.

7 La sequenza di sfruttamento passo passo

Questa sezione descrive l'attacco come una procedura ordinata. Ogni passo indica l'azione dell'attaccante e l'effetto osservabile; i confini di sicurezza attraversati sono richiamati esplicitamente per collegare la pratica all'analisi delle sezioni precedenti.

Punto d'ingresso: la command injection

La funzione di diagnostica dell'applicazione costruisce un comando di shell concatenando direttamente l'input dell'utente:

Listing 2: application/app.py (estratto rilevante)

```
target = request.form.get("target", "")
completed = subprocess.run(
    "ping -c 1 " + target,
    shell=True,
    capture_output=True,
    text=True,
    timeout=60,
)
```

L'uso di `shell=True` unito alla concatenazione di stringhe produce una classica command injection: qualunque metacarattere di shell presente in `target` viene interpretato dalla shell. [8]

Procedura di sfruttamento

Passo 1. Individuare il campo di input. L'attaccante raggiunge la dashboard su `http://HOST:8080/` e identifica il modulo di diagnostica, il cui unico parametro (`target`) è pensato per contenere un indirizzo IP o un nome host. *Confine attraversato: rete → applicazione.*

Passo 2. Confermare l'iniezione. Si invia un valore con un separatore di comando, ad esempio `127.0.0.1; id`. Se la risposta contiene, oltre all'output del `ping`, anche quello di `id`, la command injection è confermata e il passo successivo è percorribile. *Confine attraversato: input → esecuzione di codice nel container (RCE applicativa).*

Passo 3. Verificare la disponibilità del socket. Dall'esecuzione di codice ottenuta, l'attaccante controlla la presenza del piano di controllo, ad esempio con `127.0.0.1; ls -l /var/run/docker.sock` oppure `127.0.0.1; docker version`. Un esito positivo indica che il container può dialogare con il daemon. *Confine attraversato: processo → socket.*

Passo 4. Richiedere al daemon un container privilegiato. Attraverso il socket, l'attaccante chiede al daemon di avviare un nuovo container con la radice dell'host montata al suo interno. Il payload completo inviato nel campo `target` è:

Listing 3: Payload concettuale inviato nel campo `target`

```
127.0.0.1; docker run --rm -v /:/host alpine \
  sh -c 'id > /host/tmp/ESCAPE_PROOF.txt; \
         uname -a >> /host/tmp/ESCAPE_PROOF.txt; \
         head -n 1 /host/etc/shadow >> /host/tmp/ESCAPE_PROOF.txt'
```

Il separatore dopo l'indirizzo locale consente di aggiungere il comando Docker alla diagnostica prevista dall'applicazione. *Confine attraversato: client → daemon root.*

Passo 5. Agire sul filesystem dell'host. All'interno del container figlio il processo è root e vede l'intero filesystem dell'host sotto `/host`: scrive il file di prova, vi aggiunge l'output di `uname` e tenta la lettura della prima riga di `/etc/shadow`. *Confine attraversato: namespace del container → host.*

Osservazioni sul payload

Il container figlio viene creato dallo *stesso* daemon che gestisce la dashboard: non si avvia un daemon Docker dentro il container, e il termine "container figlio" indica soltanto che la richiesta di creazione proviene dal primo container. L'attacco non usa `--privileged` proprio perché non ne ha bisogno: il bind mount `/:/host` è sufficiente, e l'azione privilegiata è compiuta dal daemon per conto del chiamante. [4]

La disponibilità del client `docker` nel container, inoltre, non è un prerequisito: in sua assenza l'attaccante può parlare con l'API del daemon direttamente via HTTP sul socket, ad esempio con `curl --unix-socket /var/run/docker.sock`. La presenza del binario è una comodità, non una condizione necessaria.

L'azione dimostrativa è deliberatamente contenuta. La lettura di `/etc/shadow` coinvolge dati sensibili e non va qualificata come innocua; per una dimostrazione puramente didattica è preferibile sostituirla con la lettura di un *file sentinella* predisposto sull'host e leggibile soltanto da root, così da attestare il livello di privilegio senza raccogliere credenziali reali.

8 Impatto sulla sicurezza

L'impatto è massimo. L'attaccante passa da un'esecuzione di codice confinata in un singolo container a un controllo completo, con privilegi di root, dell'intero host e di conseguenza di tutti gli altri container che vi girano. Le conseguenze potenziali includono la lettura di qualsiasi segreto presente sul sistema (chiavi private, credenziali, file di configurazione), la manipolazione o la distruzione di dati, l'installazione di meccanismi di persistenza e il movimento laterale verso altri sistemi raggiungibili dall'host. In termini di tassonomia, l'attacco corrisponde alla tecnica *Escape to Host* (T1611) della matrice MITRE ATT&CK per i container. [6]

Occorre tuttavia distinguere l'impatto potenziale dall'evidenza effettivamente raccolta. L'output di `id` descrive l'utente del processo nel container figlio e `uname` mostra informazioni sul kernel condiviso: nessuno dei due, isolatamente, prova un escape. La presenza di un file verificato *dal lato host* dimostra invece una scrittura sul filesystem condiviso; per attestare l'accesso riservato

a root serve una prova su una risorsa con permessi appropriati, poiché una scrittura in `/tmp` è normalmente possibile anche a utenti ordinari.

9 Catena d'attacco completa (bonus)

Il laboratorio estende la dimostrazione includendo l'applicazione vulnerabile, così da mostrare l'intera progressione *applicazione vulnerabile* → *sfruttamento applicativo* → *accesso al container* → *escape* → *privilegio finale*. Analizzare la catena a stadi è istruttivo perché ciascun passaggio attraversa un confine di sicurezza distinto, in corrispondenza dei passi operativi della Sezione 7.

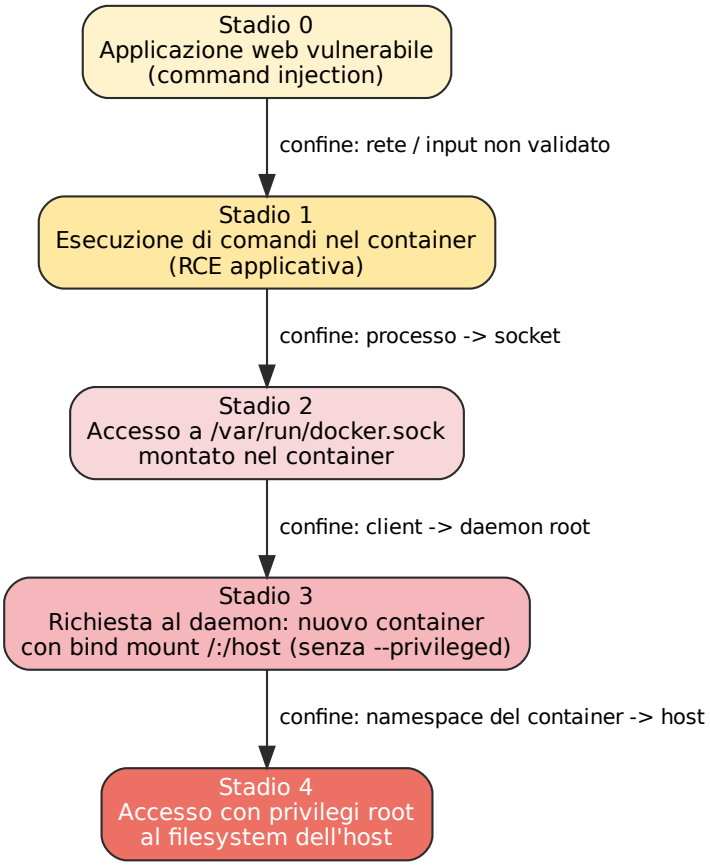


Figura 3: Gli stadi della catena d'attacco e il confine di sicurezza superato in ciascun passaggio.

Nello **Stadio 0–1** il confine superato è quello tra la rete e l'esecuzione di codice: un input non validato, trattato come parte di un comando di shell, trasforma una funzionalità legittima in esecuzione arbitraria dentro il container. Nello **Stadio 2** il confine è quello tra il processo e il piano di controllo: il processo scopre di poter dialogare con il socket del daemon. Nello **Stadio 3** si attraversa il confine tra client e daemon root: una richiesta API apparentemente ordinaria porta alla creazione di un container con la radice dell'host montata. Nello **Stadio 4**, infine, cade il confine tra container e host, e l'attaccante agisce con privilegi di root sul sistema ospitante.

La natura a stadi suggerisce anche dove intervenire. La tabella seguente riassume, per ogni stadio, il confine attraversato, una contromisura mirata e l'effetto che la sua adozione avrebbe

sull'intera catena.

Stadio	Confine	Mitigazione	Effetto sulla catena
0–1	Rete → esecuzione	Validazione dell'input; evitare `shell=True`; usare liste di argomenti	Nega l'accesso iniziale: la catena non parte.
2	Processo → socket	Non montare il socket nel container	Rimuove la primitiva di escape: l'RCE resta confinata al container.
3	Client → daemon root	Socket proxy che filtra l'API; modalità rootless	Limita o declassa le azioni ottenibili dal daemon.
4	Container → host	User namespace remapping; difesa in profondità sull'host	Riduce i privilegi risultanti anche in caso di escape.

Tabella 1: Confini, contromisure ed effetto di ciascun intervento sulla catena.

Si osserva che gli stadi non hanno la stessa importanza. Interrompere lo Stadio 2, cioè non montare il socket, è da solo sufficiente a spezzare la catena: anche con la vulnerabilità applicativa pienamente sfruttabile, l'attaccante rimarrebbe confinato nel container. Al contrario, correggere solo la vulnerabilità applicativa (Stadio 0–1) eliminerebbe *questa* via d'ingresso, ma lascerebbe la misconfigurazione pronta a essere sfruttata attraverso qualsiasi altro vettore che consenta esecuzione di codice nel container. Da qui una lezione generale: le mitigazioni più vicine alla causa strutturale (la delega di privilegio implicita nel socket) sono più robuste di quelle che bloccano un singolo punto d'ingresso.

10 Vulnerabilità note correlate

Lo scenario qui dimostrato non dipende da una CVE: non sfrutta un difetto del software, ma una configurazione che concede privilegi eccessivi. Questa distinzione è importante perché la contromisura non è "aggiornare", bensì "non concedere quel privilegio". È comunque utile collocare l'escape da misconfigurazione accanto agli escape dovuti a vulnerabilità del runtime, con cui condivide l'esito finale ma non la causa.

Un esempio recente e autorevole è **CVE-2024-21626**, nota come *Leaky Vessels*, che interessa il runtime `runc` nelle versioni da 1.0.0-rc93 fino alla 1.1.11 (CWE-403, punteggio CVSS v3.1 pari a 8.6), corretta nella 1.1.12. A causa della fuga di un descrittore di file interno, un processo del container può ritrovarsi con una directory di lavoro collocata nel filesystem dell'host, ottenendo così accesso all'host stesso. A differenza del nostro caso, qui l'escape nasce da un difetto del runtime e la mitigazione corretta è l'aggiornamento a `runc` 1.1.12 o successivo. [9] Un precedente storico analogo è **CVE-2019-5736**, in cui un container malevolo può sovrascrivere il binario `runc` dell'host sfruttando `/proc/self/exe`, ottenendo esecuzione come root sull'host. [10]

Il confronto chiarisce perché la misconfigurazione scelta sia particolarmente adatta a un laboratorio didattico: è deterministica, non richiede versioni vulnerabili specifiche, resta valida nel tempo indipendentemente dalle patch, e isola con nettezza il concetto di delega di privilegio dagli aspetti, pur interessanti, della memory safety o dei difetti di implementazione del runtime.

11 Mitigazioni e misure di hardening passo passo

Le contromisure vanno applicate come difesa in profondità, a partire dalla più vicina alla causa strutturale. Si propone la sequenza seguente, dalla priorità più alta alla più bassa.

1. **Rimuovere il socket dal container.** È la misura principale e la più semplice: eliminare la riga `- /var/run/docker.sock:/var/run/docker.sock` dal file di `compose` e ricostruire il servizio. Da sola, spezza la catena qualunque sia il vettore d'ingresso.

2. **Mediare l'accesso quando è davvero necessario.** Se il servizio deve interagire con l'API Docker, interporre un socket proxy che esponga solo gli endpoint strettamente necessari, negando per esempio la creazione di container o l'uso di bind mount. Uno strumento consolidato a questo scopo è `docker-socket-proxy` di Tecnativa. Il filtro per endpoint non va però confuso con una validazione dettagliata di ogni parametro della richiesta. [11]

3. **Ridurre i privilegi del daemon.** Valutare la modalità *rootless*, così che il daemon non giri come root e l'impatto di un eventuale abuso risulti ridotto; e, dove applicabile, il remapping degli user namespace, affinché il root del container non corrisponda al root dell'host. Si ricordi che con `userns-remap` il daemon continua comunque a operare come root e che il remapping può essere disabilitato per singoli container: non è quindi una protezione sufficiente da solo. [12, 13]

4. **Correggere la vulnerabilità applicativa.** Passare gli argomenti come lista, senza interprete di shell, ed evitare `shell=True`:

   Listing 4: Correzione dell'input nella funzione di diagnostica

   ```
   import ipaddress, subprocess

   target = request.form.get("target", "")
   ipaddress.ip_address(target)          # valida: solleva un errore se non e' un IP
   completed = subprocess.run(
       ["ping", "-c", "1", target],      # lista di argomenti, nessuna shell
       capture_output=True, text=True, timeout=60,
   )
   ```

   affiancando una validazione coerente con gli indirizzi o i nomi ammessi. [8]

5. **Applicare l'hardening complementare.** Mantenere attivi i profili di sicurezza predefiniti (seccomp, AppArmor o SELinux), evitare i container `--privileged` e le capabilities non necessarie, usare un utente dedicato, montare i volumi in sola lettura ove possibile e imporre limiti di risorse espliciti. Nessuna di queste misure, da sola, sostituisce la rimozione del socket, ma il loro insieme rende l'intera catena sensibilmente più difficile da percorrere. [1]

Se l'amministrazione del daemon avviene da remoto, SSH o TLS proteggono il canale di accesso, ma non sostituiscono la decisione su quali operazioni il chiamante possa eseguire. [3] Il CIS Docker Benchmark offre un riferimento per il riesame della configurazione, mentre NIST SP 800-190 inquadra i rischi della tecnologia container. [7, 5]

12 Verifica e riproducibilità passo passo

L'ambiente è progettato per essere ricostruito senza passaggi manuali non documentati. La procedura seguente avvia il laboratorio, esegue l'attacco, raccoglie la prova e ripulisce tutto. Va eseguita esclusivamente su una macchina Linux dedicata al laboratorio.

1. **Predisporre l'ambiente.** Costruire e avviare il servizio; al termine la dashboard è raggiungibile su `http://localhost:8080`.

```
bash scripts/setup.sh
```

2. **Registrare lo stato iniziale.** Prima della prova, annotare versione di sistema, kernel, Docker Engine, Compose e runtime, la modalità del daemon (rootful/rootless) e i controlli di sicurezza abilitati; identificare le immagini con digest espliciti, poiché il riferimento generico ad `alpine` può risolvere contenuti diversi nel tempo. Verificare che il file di prova *non* esista ancora:

```
ls -l /tmp/ESCAPE_PROOF.txt    # atteso: nessun file
```

3. **Eseguire l'attacco.** Lo script invia il payload di command injection, attende l'esecuzione del container di escape e stampa la prova.

```
bash scripts/demo.sh
```

4. **Verificare la prova dal lato host.** Ispezionare contenuto e permessi del file *direttamente sull'host del daemon*; la sua presenza, creata da un container e contenente dati letti dal filesystem dell'host, dimostra il superamento del confine container→host.

```
cat /tmp/ESCAPE_PROOF.txt
stat -c '%A %U:%G' /tmp/ESCAPE_PROOF.txt
```

5. **Controprova (isolare il ruolo del socket).** Ripetere la prova dopo aver rimosso il solo bind mount del socket, lasciando invariata la vulnerabilità applicativa: la command injection può ancora riuscire, mentre il passaggio basato sul socket deve fallire. Un secondo controllo, con l'input gestito come lista di argomenti, verifica separatamente la correzione applicativa.

6. **Ripulire l'ambiente.** Rimuovere container, immagine, volumi e file di prova.

```
bash scripts/cleanup.sh
```

Per rendere inequivocabile l'evidenza conviene documentare il file di prova prima e dopo l'esecuzione, assegnargli un identificatore univoco che lo distingua da residui di sessioni precedenti e, se si usa un file sentinella protetto, annotarne i permessi iniziali che ne rendono significativa la lettura.

13 Conclusioni

Il laboratorio mostra che l'isolamento dei container è una proprietà fragile, che dipende dalla configurazione tanto quanto dal software. Una singola, apparentemente innocua, scelta — montare il socket del daemon — è sufficiente a trasformare un'esecuzione di codice confinata in una compromissione completa dell'host, senza bisogno di alcun exploit. L'analisi a stadi della catena d'attacco, resa esplicita nei passi operativi di questa relazione, evidenzia che la mitigazione più efficace è quella che agisce sulla causa strutturale, ossia la delega implicita di privilegio: rimuovere il socket spezza la catena a prescindere da come l'attaccante sia riuscito a entrare nel container. È un'illustrazione concreta del principio del privilegio minimo e del valore della difesa in profondità. La validità sperimentale della relazione dipende infine dalle evidenze raccolte sul sistema che ospita Docker e dalla distinzione fra accesso dimostrato e compromissione potenziale.

Riferimenti bibliografici

[1] OWASP, *Docker Security Cheat Sheet*.
`https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html`

[2] Docker, *Docker daemon attack surface / Docker Engine security*.
`https://docs.docker.com/engine/security/`

[3] Docker, *Protect the Docker daemon socket*.
`https://docs.docker.com/engine/security/protect-access/`

[4] Docker, *Bind mounts*.
`https://docs.docker.com/engine/storage/bind-mounts/`

[5] NIST, *SP 800-190 Application Container Security Guide*.
`https://doi.org/10.6028/NIST.SP.800-190`

[6] MITRE ATT&CK, *Escape to Host (T1611)*.
`https://attack.mitre.org/techniques/T1611/`

[7] Center for Internet Security, *CIS Docker Benchmark*.
`https://www.cisecurity.org/benchmark/docker`

[8] Python Software Foundation, *subprocess — Subprocess management*.
`https://docs.python.org/3/library/subprocess.html`

[9] CVE-2024-21626 — *runc container breakout (Leaky Vessels)*, advisory GHSA-xr7r-f8xq-vfvv.
`https://github.com/opencontainers/runc/security/advisories/GHSA-xr7r-f8xq-vfvv`

[10] CVE-2019-5736 — *runc /proc/self/exe overwrite*.
`https://nvd.nist.gov/vuln/detail/CVE-2019-5736`

[11] Tecnativa, *docker-socket-proxy*.
`https://github.com/Tecnativa/docker-socket-proxy`

[12] Docker, *Rootless mode*.
`https://docs.docker.com/engine/security/rootless/`

[13] Docker, *Isolate containers with a user namespace.*
`https://docs.docker.com/engine/security/userns-remap/`